

ບົດທີ 2: ຂະບວນການພັດທະນາຊອບແວ (Software Development Processes)

ຫຼັງຈາກທີ່ເຮົາໄດ້ຮຽນຮູ້ແລ້ວວ່າ ວິສະວະກຳຊອບແວຄືການນຳໃຊ້ວິທີການທີ່ເປັນລະບົບ, ຄຳຖາມຕໍ່ມາກໍຄື: "ລະບົບນັ້ນມີ ຫ້າຕາເປັນແນວໃດ?". ຄຳຕອບກໍຄື **ວົງຈອນຊີວິດການພັດທະນາຊອບແວ (Software Development Life Cycle - SDLC)**.

2.1 ພາບລວມຂອງວົງຈອນຊີວິດການພັດທະນາຊອບແວ (SDLC)

ກ. SDLC ແມ່ນຫຍັງ?

ວົງຈອນຊີວິດການພັດທະນາຊອບແວ (SDLC) ຄືກອບ (Framework) ທີ່ກຳນົດຂັ້ນຕອນ ຫຼື ກົດຈະກຳຕ່າງໆ ທີ່ຕ້ອງ ເຮັດໃນໂຄງການພັດທະນາຊອບແວ, ຕັ້ງແຕ່ຈຸດເລີ່ມຕົ້ນໄປຈົນເຖິງຈຸດສິ້ນສຸດ. ມັນຄືແຜນຜັງທີ່ຊ່ວຍໃຫ້ທີມງານເຫັນ ພາບລວມວ່າຈະຕ້ອງເຮັດຫຍັງແດ່, ເຮັດເມື່ອໃດ, ແລະ ເຮັດແນວໃດ ເພື່ອໃຫ້ສາມາດສ້າງຊອບແວໄດ້ຢ່າງມີລະບຽບ ແລະ ມີຄຸນນະພາບ.

ໃຫ້ລອງຄິດພາບ SDLC ເປັນ **"ສູດການເຮັດອາຫານ"**. ກ່ອນທີ່ເຮົາຈະໄດ້ອາຫານທີ່ແຊບໜຶ່ງຈານ (ຊອບແວ), ເຮົາຕ້ອງ ຜ່ານຂັ້ນຕອນຕ່າງໆ ເຊັ່ນ: ການວາງແຜນ (ຈະເຮັດຫຍັງ), ການຊື້ສ່ວນປະກອບ (ເກັບກຳຄວາມຕ້ອງການ), ການຂົ່ວ-ຕົ້ມ- ແກງ (ການຂຽນໂຄດ), ແລະ ການຈັດຈານ (ການນຳໄປໃຊ້ງານ).

ທຸກໆໂຄງການຊອບແວ, ບໍ່ວ່າຈະໃຫຍ່ຫຼືນ້ອຍ, ລ້ວນແຕ່ຕ້ອງຜ່ານກົດຈະກຳຫຼັກໆເຫຼົ່ານີ້. ສິ່ງທີ່ແຕກຕ່າງກັນໃນແຕ່ລະ "ໂມເດລ" (ເຊັ່ນ Waterfall ຫຼື Agile) ກໍຄື **ວິທີການຈັດລຽງ ແລະ ວິທີການເຮັດຊ້ຳ** ຂັ້ນຕອນເຫຼົ່ານີ້.

ຂ. ກົດຈະກຳຫຼັກ 6 ຢ່າງໃນ SDLC (Core Activities of SDLC)

ເຖິງແມ່ນວ່າຈະມີຫຼາຍໂມເດລ, ແຕ່ແຜນຫຼັກຂອງ SDLC ປະກອບດ້ວຍ 6 ກົດຈະກຳສຳຄັນດັ່ງນີ້:

1. ການວາງແຜນ (Planning)

- ຄຳຖາມຫຼັກ: ເຮົາຈະສ້າງຫຍັງ? ແລະ ມັນເປັນໄປໄດ້ບໍ່?
- ກົດຈະກຳ:
 - ການສຶກສາຄວາມເປັນໄປໄດ້ (Feasibility Study):** ວິເຄາະວ່າໂຄງການນີ້ຄຸ້ມຄ່າທີ່ຈະເຮັດບໍ່ (ທາງ ເສດຖະກິດ), ເຮົາສາມາດເຮັດໄດ້ບໍ່ (ທາງເຕັກນິກ), ແລະ ເຮັດໄດ້ໃນເວລາທີ່ກຳນົດບໍ່ (ທາງການດຳເນີນ ງານ).
 - ກຳນົດຂອບເຂດຂອງໂຄງການ (Project Scope):** ກຳນົດໃຫ້ຊັດເຈນວ່າຊອບແວທີ່ຈະສ້າງນັ້ນ ຈະເຮັດ ຫຍັງໄດ້ແດ່ ແລະ ຈະບໍ່ເຮັດຫຍັງ.
 - ການວາງແຜນໂຄງການເບື້ອງຕົ້ນ:** ປະເມີນເວລາ, ງົບປະມານ, ແລະ ຊັບພະຍາກອນທີ່ຕ້ອງໃຊ້.
- ຜົນລັບ (Output):** ເອກະສານແຜນການໂຄງການ (Project Plan).

2. ການເກັບກຳ ແລະ ວິເຄາະຄວາມຕ້ອງການ (Requirement Gathering and Analysis)

- ຄຳຖາມຫຼັກ: ຜູ້ໃຊ້ ແລະ ຜູ້ມີສ່ວນຮ່ວມຕ້ອງການໃຫ້ຊອບແວເຮັດຫຍັງແດ່?

- **ກິດຈະກຳ:**
 - ລົມກັບລູກຄ້າ, ຜູ້ໃຊ້, ແລະ ຜູ້ມີສ່ວນຮ່ວມ (Stakeholders) ເພື່ອເກັບກຳຄວາມຕ້ອງການທັງໝົດ.
 - ວິເຄາະ ແລະ ຈັດລຳດັບຄວາມສຳຄັນຂອງຄວາມຕ້ອງການ.
 - ແຍກປະເພດຄວາມຕ້ອງການ (Functional vs. Non-functional).
- **ຜົນລັບ (Output):** ເອກະສານກຳນົດຄວາມຕ້ອງການຊອບແວ (Software Requirement Specification - SRS).

3. ການອອກແບບ (Design)

- **ຄຳຖາມຫຼັກ:** ເຮົາຈະສ້າງແນວໃດ?
- **ກິດຈະກຳ:** ຂັ້ນຕອນນີ້ຄືການແປ "ຄວາມຕ້ອງການ" ໃຫ້ກາຍເປັນ "ແບບແຜນ" ຂອງຊອບແວ. ມັນແບ່ງອອກເປັນ 2 ລະດັບ:
 - **ການອອກແບບລະດັບສູງ (High-level Design):** ການອອກແບບສະຖາປັດຕະຍະກຳລວມຂອງລະບົບ, ການເລືອກເທັກໂນໂລຢີ, ແລະ ການກຳນົດວ່າລະບົບຈະມີໂມດູນຫຍັງແດ່ ແລະ ເຊື່ອມຕໍ່ກັນແນວໃດ.
 - **ການອອກແບບລະດັບຕໍ່າ (Low-level Design):** ການອອກແບບລາຍລະອຽດພາຍໃນແຕ່ລະໂມດູນ, ເຊັ່ນ: ການອອກແບບ Class, ຖານຂໍ້ມູນ, ແລະ ຂັ້ນຕອນວິທີ (Algorithms) ທີ່ຈະໃຊ້.
- **ຜົນລັບ (Output):** ເອກະສານການອອກແບບ (Design Document) ທີ່ປະກອບດ້ວຍ UML Diagrams, Schema ຂອງຖານຂໍ້ມູນ, ແລະ ອື່ນໆ.

4. ການພັດທະນາ ຫຼື ການຂຽນໂຄດ (Development / Coding)

- **ຄຳຖາມຫຼັກ:** ລົງມືສ້າງຕາມແບບແຜນ.
- **ກິດຈະກຳ:**
 - ນັກພັດທະນາ (Developers) ນຳເອົາເອກະສານການອອກແບບມາແປເປັນ Source Code ທີ່ສາມາດເຮັດວຽກໄດ້.
 - ຂັ້ນຕອນນີ້ອາດຈະລວມເຖິງການຂຽນ Unit Test ເພື່ອທົດສອບໂຄດໃນລະດັບພື້ນຖານ.
- **ຜົນລັບ (Output):** Source Code ຂອງໂປຣແກຣມ.

5. ການທົດສອບ (Testing)

- **ຄຳຖາມຫຼັກ:** ຊອບແວທີ່ສ້າງຂຶ້ນ ເຮັດວຽກໄດ້ຖືກຕ້ອງ ແລະ ຕອບໂຈດຄວາມຕ້ອງການບໍ່?
- **ກິດຈະກຳ:**
 - ທີມງານຮັບປະກັນຄຸນນະພາບ (QA Team) ຈະທຳການທົດສອບຊອບແວຢ່າງເຂັ້ມງວດ ເພື່ອຊອກຫາ Bug ແລະ ຂໍ້ຜິດພາດຕ່າງໆ.
 - ມີການທົດສອບຫຼາຍລະດັບ ເຊັ່ນ: Integration Testing (ທົດສອບການເຊື່ອມຕໍ່ລະຫວ່າງໂມດູນ) ແລະ System Testing (ທົດສອບລະບົບທັງໝົດ).
 - ກວດສອບວ່າຊອບແວເຮັດວຽກໄດ້ຕາມທີ່ລະບຸໄວ້ໃນເອກະສານ SRS.
- **ຜົນລັບ (Output):** ລາຍງານຜົນການທົດສອບ (Test Report) ແລະ ຊອບແວທີ່ມີຄຸນນະພາບ.

6. ການນຳໄປໃຊ້ງານ ແລະ ການບຳລຸງຮັກສາ (Deployment and Maintenance)

- **ຄຳຖາມຫຼັກ:** ເຮັດແນວໃດໃຫ້ຜູ້ໃຊ້ສາມາດເລີ່ມໃຊ້ງານໄດ້? ແລະ ຈະດູແລຮັກສາມັນແນວໃດ?
- **ກິດຈະກຳ:**
 - **Deployment:** ການຕິດຕັ້ງຊອບແວເທິງສະພາບແວດລ້ອມຈິງ (Production Server) ເພື່ອໃຫ້ຜູ້ໃຊ້ສາມາດເລີ່ມໃຊ້ງານໄດ້.
 - **Maintenance:** ຫຼັງຈາກເປີດໃຊ້ງານແລ້ວ, ວຽກຍັງບໍ່ຈົບ. ຈະຕ້ອງມີການດູແລຮັກສາຕໍ່ເນື່ອງ ເຊິ່ງລວມມີ:
 - **ການແກ້ໄຂ Bug:** ແກ້ໄຂຂໍ້ຜິດພາດທີ່ພົບເຫັນຫຼັງຈາກການໃຊ້ງານຈິງ.
 - **ການອັບເກຣດ:** ເພີ່ມ Features ໃໝ່ໆ ຕາມຄວາມຕ້ອງການທີ່ປ່ຽນໄປ.
 - **ການປັບປຸງ:** ປັບປຸງປະສິດທິພາບ ຫຼື ຄວາມປອດໄພຂອງລະບົບ.
- **ຜົນລັບ (Output):** ຊອບແວທີ່ໃຊ້ງານໄດ້ຈິງ ແລະ ມີການອັບເດດຢ່າງຕໍ່ເນື່ອງ.

ຕົວຢ່າງ:

ໃຫ້ລອງຄິດພາບການສ້າງ "ແອັບ ສັງອາຫານ" ຂອງຮ້ານອາຫານແຫ່ງໜຶ່ງ:

1. **Planning:** ເຈົ້າຂອງຮ້ານຕັດສິນໃຈວ່າຈະສ້າງແອັບ, ປະເມີນງົບປະມານ 100,000,000 ກີບ, ແລະ ຕັ້ງເປົ້າໃຫ້ສຳເລັດພາຍໃນ 3 ເດືອນ.
2. **Requirements:** ທີມງານໄປສຳພາດເຈົ້າຂອງຮ້ານ ແລະ ພະນັກງານ ເພື່ອຮູ້ວ່າຕ້ອງການຫຍັງແດ່ (ເບິ່ງເມນູ, ສັງອາຫານ, ຈ່າຍເງິນ, ໃຫ້ຄະແນນ).
3. **Design:** ທີມງານອອກແບບໜ້າຕາຂອງແອັບ, ອອກແບບຖານຂໍ້ມູນເພື່ອເກັບຂໍ້ມູນເມນູ ແລະ ລາຍການສັ່ງຊື້.
4. **Development:** ໂປຣແກຣມເມີລົງມືຂຽນໂຄດສ້າງໜ້າຕ່າງໆ ແລະ ເຊື່ອມຕໍ່ກັບຖານຂໍ້ມູນ.
5. **Testing:** ທີມ QA ລອງກົດສັ່ງອາຫານ, ລອງຈ່າຍເງິນ, ແລະ ພະຍາຍາມເຮັດໃຫ້ແອັບ ຂັດຂ້ອງເພື່ອຫາ Bug.
6. **Deployment & Maintenance:** ເມື່ອທຸກຢ່າງຮຽບຮ້ອຍ, ກໍເອົາແອັບ ຂຶ້ນ App Store ໃຫ້ລູກຄ້າດາວໂຫຼດ. ຫຼັງຈາກນັ້ນ 1 ເດືອນ, ເຈົ້າຂອງຮ້ານຕ້ອງການເພີ່ມ Feature "ໂປຣໂມຊັນ", ທີມງານກໍຈະເຂົ້າສູ່ຂັ້ນຕອນການບຳລຸງຮັກສາ.

ບົດສະຫຼຸບ:

SDLC ຄືຫົວໃຈຂອງການພັດທະນາຊອບແວຢ່າງເປັນລະບົບ. ການເຂົ້າໃຈ 6 ຂັ້ນຕອນນີ້ ເປັນພື້ນຖານທີ່ຈຳເປັນທີ່ສຸດ ເພາະທຸກໆ Software Process Model ທີ່ເຮົາຈະໄດ້ຮຽນຕໍ່ໄປ (ເຊັ່ນ Waterfall, Agile) ລ້ວນແລ້ວແຕ່ເປັນພຽງ "ວິທີການຈັດການ" ກິດຈະກຳທັງ 6 ຢ່າງນີ້ໃນຮູບແບບທີ່ແຕກຕ່າງກັນເທົ່ານັ້ນ.

2.2 ໂມເດລແບບດັ້ງເດີມ (ຮຽນຮູ້ໃນທາງປະຫວັດສາດ): Waterfall Model

ຫຼັງຈາກທີ່ເຮົາໄດ້ເຫັນພາບລວມຂອງ SDLC ແລ້ວ, ບາດນີ້ເຮົາມາລິງເລິກໃນ "ໂມເດລ" ທຳອິດທີ່ເປັນຈຸດເລີ່ມຕົ້ນຂອງການພັດທະນາຊອບແວຢ່າງເປັນລະບົບ. ໂມເດລນີ້ອາດຈະບໍ່ແມ່ນວິທີທີ່ດີທີ່ສຸດໃນປັດຈຸບັນ, ແຕ່ການສຶກສາມັນຄືການຮຽນຮູ້ຈາກປະຫວັດສາດທີ່ສຳຄັນ.

ກ. Waterfall Model ແມ່ນຫຍັງ?

Waterfall Model (ໂມເດລນ້ຳຕົກຕາດ) ຄືໂມເດລຂະບວນການພັດທະນາຊອບແວແບບຕັ້ງເດີມ ແລະ ເປັນທາງການທຳອິດ. ຊື່ຂອງມັນມາຈາກການປຽບທຽບກັບນ້ຳຕົກຕາດ ທີ່ນ້ຳຈະ ໄຫຼລົງຈາກຂັ້ນເທິງສູ່ຂັ້ນລຸ່ມພຽງທາງດຽວ ແລະ ບໍ່ມີທາງໄຫຼກັບຄືນ.

ແນວຄິດຫຼັກຂອງ Waterfall Model ຄື ການພັດທະນາແບບເປັນລຳດັບຂັ້ນ (Sequential). ໝາຍຄວາມວ່າ, ແຕ່ລະຂັ້ນຕອນຂອງ SDLC ຕ້ອງຖືກເຮັດໃຫ້ "ສຳເລັດຢ່າງສົມບູນ 100%" ກ່ອນ ຈຶ່ງຈະສາມາດເລີ່ມຂັ້ນຕອນຕໍ່ໄປໄດ້. ເມື່ອຜ່ານຂັ້ນຕອນໃດໜຶ່ງໄປແລ້ວ, ຈະບໍ່ມີການກັບຄືນໄປແກ້ໄຂຂັ້ນຕອນກ່ອນໜ້າອີກ.

ຂ. ຂັ້ນຕອນຂອງ Waterfall Model

ໂມເດລນີ້ຈະນຳເອົາກິດຈະກຳຫຼັກຂອງ SDLC ມາຈັດລຽງຕໍ່ກັນເປັນເສັ້ນຊື່:

1. **ການເກັບກຳຄວາມຕ້ອງການ (Requirements):** ທຸກໆຄວາມຕ້ອງການຂອງລະບົບຈະຖືກເກັບກຳ, ວິເຄາະ, ແລະ ບັນທຶກໄວ້ໃນເອກະສານ SRS ຢ່າງລະອຽດ. ເມື່ອເອກະສານນີ້ຖືກຮັບຮອງ (Sign-off) ແລ້ວ, ມັນຈະຖືກ "ລືອກ" ແລະ ບໍ່ສາມາດປ່ຽນແປງໄດ້.
2. **ການອອກແບບ (Design):** ທີມງານຈະນຳເອົາເອກະສານ SRS ທີ່ຖືກລືອກໄວ້ມາສ້າງເປັນແບບແປນຂອງລະບົບທັງໝົດ, ທັງໃນລະດັບສູງ (ສະຖາປັດຕະຍະກຳ) ແລະ ລະດັບຕໍ່າ (Class, Database). ເມື່ອອອກແບບສຳເລັດ, ເອກະສານອອກແບບກໍຈະຖືກລືອກ.
3. **ການພັດທະນາ (Implementation):** ໂປຣແກຣມເມີຈະຂຽນໂຄດຕາມແບບແປນທີ່ຖືກລືອກໄວ້ເທົ່ານັ້ນ. ບໍ່ມີການເພີ່ມ Feature ໃໝ່ ຫຼື ປ່ຽນແປງການອອກແບບໃນຂັ້ນຕອນນີ້.
4. **ການທົດສອບ (Testing):** ຫຼັງຈາກທີ່ຂຽນໂຄດທັງໝົດແລ້ວ, ທີມ QA ຈະນຳເອົາຊອບແວທັງລະບົບມາທົດສອບ ເພື່ອຊອກຫາ Bug.
5. **ການນຳໄປໃຊ້ງານ (Deployment):** ຫຼັງຈາກທີ່ແກ້ໄຂ Bug ທັງໝົດແລ້ວ, ຊອບແວຈະຖືກຕິດຕັ້ງໃຫ້ລູກຄ້າໃຊ້ງານ.
6. **ການບຳລຸງຮັກສາ (Maintenance):** ທີມງານຈະຄອບແກ້ໄຂບັນຫາທີ່ພົບໃນພາຍຫຼັງ ຫຼື ປັບປຸງລະບົບເລັກນ້ອຍ.

ຄ. ຈຸດແຂງ (Strengths) - ເປັນຫຍັງມັນຈຶ່ງເຄີຍເປັນທີ່ນິຍົມ?

ເຖິງແມ່ນວ່າຈະຫຼ້າສະໄໝ, ແຕ່ Waterfall Model ກໍມີປະໂຫຍດໃນບາງສະຖານະການ:

- **ງ່າຍຕໍ່ການເຂົ້າໃຈ ແລະ ບໍລິຫານຈັດການ:** ຂັ້ນຕອນຊັດເຈນ, ເປັນເສັ້ນຊື່, ເຮັດໃຫ້ງ່າຍຕໍ່ການວາງແຜນ ແລະ ຕິດຕາມຄວາມຄືບໜ້າ.
- **ເໝາະສົມກັບໂຄງການທີ່ຄວາມຕ້ອງການຄົງທີ່:** ມັນຈະເຮັດວຽກໄດ້ດີທີ່ສຸດ ກັບໂຄງການທີ່ຄວາມຕ້ອງການທັງໝົດຊັດເຈນທີ່ສຸດ, ຮູ້ລ່ວງໜ້າທັງໝົດ, ແລະ ຮັບປະກັນວ່າຈະບໍ່ມີການປ່ຽນແປງຢ່າງເດັດຂາດ.
- **ມີເອກະສານຄົບຖ້ວນ:** ເພາະແຕ່ລະຂັ້ນຕອນຕ້ອງສ້າງເອກະສານໃຫ້ສຳເລັດກ່ອນ, ເຮັດໃຫ້ມີການບັນທຶກທຸກຢ່າງໄວ້ເປັນຢ່າງດີ.

ງ. ຈຸດອ່ອນ (Weaknesses) - ເປັນຫຍັງໂລກຈຶ່ງຕ້ອງປ່ຽນແປງ?

ຈຸດອ່ອນຂອງ Waterfall Model ຄືເຫດຜົນຫຼັກທີ່ພາໃຫ້ເກີດແນວຄິດ Agile. ບັນຫາທີ່ສຳຄັນທີ່ສຸດຄື:

- **ແຂງກະດ້າງ, ບໍ່ຍືດຍຸ່ນ (Rigid):** ນີ້ຄືບັນຫາໃຫຍ່ທີ່ສຸດ. ໃນໂລກຄວາມເປັນຈິງ, ຄວາມຕ້ອງການຂອງລູກຄ້າມີການປ່ຽນແປງຕະຫຼອດເວລາ. Waterfall Model ບໍ່ສາມາດຮອງຮັບການປ່ຽນແປງໄດ້ເລີຍ.
- **ໄດ້ຮັບຄໍາຄິດເຫັນຊ້າເກີນໄປ (Late Feedback):** ລູກຄ້າ ຫຼື ຜູ້ໃຊ້ຈະບໍ່ໄດ້ເຫັນ ຫຼື ຈັບຕ້ອງຜະລິດຕະພັນເລີຍ ຈົນກວ່າຈະຮອດຂັ້ນຕອນສຸດທ້າຍ. ຖ້າສິ່ງທີ່ສ້າງຂຶ້ນບໍ່ແມ່ນສິ່ງທີ່ລູກຄ້າຕ້ອງການແທ້ໆ, ກໍຈະຮູ້ເມື່ອສາຍໄປແລ້ວ.
- **ຄວາມສ່ຽງສູງ (High Risk):** ຄວາມສ່ຽງທັງໝົດຂອງໂຄງການຈະຖືກກອງລວມກັນໄວ້ໃນຕອນທ້າຍ. ອາດຈະໃຊ້ເວລາ 1 ປີ ເພື່ອສ້າງສິ່ງທີ່ຜິດພາດທັງໝົດ.
- **ການແກ້ໄຂຂໍ້ຜິດພາດມີຄ່າໃຊ້ຈ່າຍສູງ:** ຖ້າທີມ QA ພົບຂໍ້ຜິດພາດທີ່ຮ້າຍແຮງໃນການ "ອອກແບບ" ໃນຂະນະທີ່ກຳລັງ "ທົດສອບ", ການກັບໄປແກ້ໄຂຈະເປັນເລື່ອງທີ່ຍາກລຳບາກ ແລະ ໃຊ້ຄ່າໃຊ້ຈ່າຍມະຫາສານ.

ຕົວຢ່າງ: ໂຄງການສ້າງ "ລະບົບຈັດການຫ້ອງສະໝຸດ" ດ້ວຍ Waterfall Model

1. **Requirements (2 ເດືອນ):** ທີມງານໄປສຳພາດນາຍພັດທະນາ ແລະ ຂຽນເອກະສານ SRS ໝາ 100 ໜ້າ ທີ່ລະບຸທຸກຢ່າງ (ການຢືມ, ການສົ່ງຄືນ, ການຄົ້ນຫາ). ເມື່ອສຳເລັດ, ທຸກຄົນເຊັນຮັບຮອງ. **ເອກະສານນີ້ຖືກລືອກ!**
2. **Design (3 ເດືອນ):** ສະຖາປະນິກຊອບແວ ອອກແບບຖານຂໍ້ມູນ, ແຕ້ມ UML Diagrams ທັງໝົດໂດຍອີງໃສ່ SRS ເທົ່ານັ້ນ. **ແບບແປນຖືກລືອກ!**
3. **Implementation (6 ເດືອນ):** ໂປຣແກຣມເມີ 10 ຄົນ ຂຽນໂຄດຕາມແບບແປນຢ່າງເຄັ່ງຄັດ.
4. **Testing (3 ເດືອນ):** ຫຼັງຈາກພັດທະນາໄປແລ້ວເກືອບ 1 ປີ, ທີມ QA ເອົາລະບົບທັງໝົດມາທົດສອບ.
 - **ບັນຫາເກີດຂຶ້ນ!** ທີມ QA ພົບວ່າ: ໃນເອກະສານ SRS ລົມລະບຸເຖິງລະບົບການຄິດໄລ່ຄ່າປັບໄໝສຳລັບການສົ່ງໜັງສືຊື້. ນີ້ຄື Feature ທີ່ສຳຄັນທີ່ສຸດທີ່ຕົກຫຼົ່ນໄປ!
5. **ຜົນລັບ:** ໂຄງການຕ້ອງຢຸດສະງັກ, ຕ້ອງມີການປະຊຸມໃຫຍ່, ແລະ ອາດຈະຕ້ອງກັບໄປເລີ່ມຂະບວນການອອກແບບ ແລະ ຂຽນໂຄດໃໝ່ໃນບາງສ່ວນ, ເຮັດໃຫ້ໂຄງການຊັກຊ້າ ແລະ ງົບປະມານບານປາຍ.

ບົດສະຫຼຸບ:

Waterfall Model ຄືບາດກ້າວທຳອິດທີ່ສຳຄັນໃນການນຳເອົາ "ລະບຽບ" ເຂົ້າມາສູ່ "ຄວາມວຸ່ນວາຍ" ຂອງການພັດທະນາຊອບແວ. ມັນສອນໃຫ້ພວກເຮົາຮູ້ຈັກຄວາມສຳຄັນຂອງແຕ່ລະຂັ້ນຕອນ. ແຕ່ດ້ວຍລັກສະນະທີ່ແຂງກະດ້າງ ແລະ ບໍ່ສາມາດຮອງຮັບການປ່ຽນແປງໄດ້, ມັນຈຶ່ງເປັນເຫດຜົນທີ່ໂຄງການຊອບແວສ່ວນໃຫຍ່ໃນໂລກປັດຈຸບັນທີ່ເຕັມໄປດ້ວຍການປ່ຽນແປງ. ບັນຫາເຫຼົ່ານີ້ໄດ້ກາຍເປັນແຮງຜັກດັນໃຫ້ເກີດໂມເດລທີ່ຍືດຍຸ່ນກວ່າ, ເຊິ່ງເຮົາຈະໄດ້ຮຽນໃນຫົວຂໍ້ຕໍ່ໄປ.

2.3 ໂມເດລສະໄໝໃໝ່ທີ່ຕ້ອງຮູ້

ຈາກບົດຮຽນຂອງ Waterfall Model, ວິສະວະກອນຊອບແວທົ່ວໂລກໄດ້ຮັບຮູ້ວ່າ ບັນຫາໃຫຍ່ທີ່ສຸດຄື "ການປ່ຽນແປງ" ແລະ "ການຂາດຄໍາຄິດເຫັນ (Feedback)". ດັ່ງນັ້ນ, ຈຶ່ງໄດ້ມີການພັດທະນາໂມເດລໃໝ່ໆ ຂຶ້ນມາ ເພື່ອແກ້ໄຂຈຸດອ່ອນເຫຼົ່ານີ້. ໂມເດລເຫຼົ່ານີ້ຄືບາດກ້າວສຳຄັນທີ່ນຳໄປສູ່ແນວຄິດ Agile ທີ່ເຮົາໃຊ້ກັນໃນປັດຈຸບັນ.

ກ. ໂມເດລການພັດທະນາແບບເພີ່ມສ່ວນ (Incremental Model)

- **ແນວຄິດຫຼັກແມ່ນຫຍັງ?**
 - ແທນທີ່ຈະສ້າງຊອບແວທັງໝົດໃຫ້ສຳເລັດໃນເທື່ອດຽວ, ໂມເດລນີ້ຈະແບ່ງລະບົບອອກເປັນສ່ວນນ້ອຍໆ ທີ່ເອີ້ນວ່າ "Increments" (ສ່ວນເພີ່ມ). ທີມງານຈະພັດທະນາ ແລະ ສົ່ງມອບແຕ່ລະ

Increment ໃຫ້ສໍາເລັດເປັນສ່ວນໆ. ແຕ່ລະ Increment ທີ່ສິ່ງມອບຈະເປັນ **ຊອບແວທີ່ໃຊ້ງານໄດ້ຈິງ** ແລະ ມີ Features ເພີ່ມຂຶ້ນຈາກສ່ວນກ່ອນໜ້າ.

- **ການປຸງບທຽບ:** ຄືກັບການສ້າງບ້ານເທື່ອລະຫ້ອງ. ເຮົາສ້າງ "ຫ້ອງຮັບແຂກ" (Increment 1) ໃຫ້ສໍາເລັດ, ສາມາດເຂົ້າໄປນັ່ງໄດ້. ຫຼັງຈາກນັ້ນ, ຈຶ່ງສ້າງ "ຫ້ອງຄົວ" (Increment 2) ເພີ່ມເຂົ້າໄປ.
- **ຂັ້ນຕອນການເຮັດວຽກ:**
 - ແຕ່ລະ Increment ຈະຜ່ານຂະບວນການທີ່ຄ້າຍຄືກັບ "Mini-Waterfall" (Requirements -> Design -> Code -> Test).
 - ຫຼັງຈາກ Increment ທີ 1 ສໍາເລັດ ແລະ ສິ່ງມອບແລ້ວ, ທີມງານກໍຈະເລີ່ມເຮັດ Increment ທີ 2 ໂດຍນໍາມາເຊື່ອມຕໍ່ກັບສ່ວນທໍາອິດ, ແລະ ເຮັດແບບນີ້ໄປເລື້ອຍໆ ຈົນກວ່າຈະຄົບທຸກ Features.
- **ມັນແກ້ໄຂບັນຫາຂອງ Waterfall ແນວໃດ?**
 - **ຫຼຸດຄວາມສ່ຽງ ແລະ ໄດ້ຮັບ Feedback ໄວຂຶ້ນ:** ລູກຄ້າບໍ່ຕ້ອງລໍຖ້າຈົນຈົບໂຄງການ. ພວກເຂົາຈະໄດ້ເຫັນ ແລະ ລອງໃຊ້ຊອບແວສ່ວນທໍາອິດ (Increment 1) ຕັ້ງແຕ່ທໍາອິດ. ຖ້າມີສິ່ງທີ່ບໍ່ຖືກຕ້ອງ, ກໍສາມາດປັບປ່ຽນແຜນສໍາລັບ Increment ຕໍ່ໄປໄດ້.
 - **ສິ່ງມອບຄຸນຄ່າໄດ້ໄວ:** ຜະລິດຕະພັນທີ່ມີ Features ຫຼັກ ສາມາດອອກສູ່ຕະຫຼາດໄດ້ໄວກວ່າ.
- **ເໝາະສົມກັບໂຄງການແບບໃດ?**
 - ໂຄງການທີ່ສາມາດແບ່ງ Features ອອກເປັນສ່ວນໆ ໄດ້ຢ່າງຊັດເຈນ.
 - ໂຄງການທີ່ຕ້ອງການປ່ອຍຜະລິດຕະພັນທີ່ມີໜ້າທີ່ການໃຊ້ງານຫຼັກອອກໄປກ່ອນ ເພື່ອຕອບສະໜອງຄວາມຕ້ອງການຂອງຕະຫຼາດ.
- **ຕົວຢ່າງ:** ໂຄງການສ້າງ "ລະບົບ E-commerce"
 - **Increment 1 (Core Shopping):** ສ້າງໜ້າທີ່ການໃຊ້ງານພື້ນຖານທີ່ສຸດ: ລູກຄ້າສາມາດເບິ່ງລາຍການສິນຄ້າ ແລະ ກິດເພີ່ມສິນຄ້າໃສ່ກະຕ່າໄດ້. (ສິ່ງມອບໃຫ້ໃຊ້ກ່ອນ)
 - **Increment 2 (Checkout System):** ເພີ່ມລະບົບການຊໍາລະເງິນ ແລະ ການຈັດການທີ່ຢູ່ສິນຄ້າ.
 - **Increment 3 (User Accounts & Reviews):** ເພີ່ມລະບົບສະມາຊິກ, ປະຫວັດການສັ່ງຊື້, ແລະ ການໃຫ້ຄະແນນສິນຄ້າ.

ຂ. ໂມເດລການພັດທະນາແບບເຮັດຊ້ໍາ (Iterative Model)

- **ແນວຄິດຫຼັກແມ່ນຫຍັງ?**
 - ໂມເດລນີ້ເນັ້ນໃສ່ການ **"ປັບປຸງ ແລະ ພັດທະນາ"**. ທີມງານຈະເລີ່ມຕົ້ນດ້ວຍການສ້າງ **"ໂຄງຮ່າງ"** ຂອງລະບົບທັງໝົດແບບງ່າຍໆ ກ່ອນ. ຫຼັງຈາກນັ້ນ, ໃນແຕ່ລະຮອບ (Iteration), ທີມງານຈະກັບມາປັບປຸງ ແລະ ເພີ່ມລາຍລະອຽດໃຫ້ກັບໂຄງຮ່າງນັ້ນ ຈົນກວ່າມັນຈະກາຍເປັນລະບົບທີ່ສົມບູນ.
 - **ການປຸງບທຽບ:** ຄືກັບການແຕ້ມຮູບຄືນ. ເຮົາເລີ່ມຈາກການຮ່າງໂຄງໜ້າທັງໝົດແບບງ່າຍໆ (Iteration 1). ຫຼັງຈາກນັ້ນ, ເຮົາກໍກັບມາເພີ່ມລາຍລະອຽດຂອງຕາ, ດັງ, ປາກ (Iteration 2). ແລ້ວກໍກັບມາລົງສີ, ແສງ, ເງົາ ໃຫ້ທົ່ວທັງຮູບ (Iteration 3).
- **ຂັ້ນຕອນການເຮັດວຽກ:**
 - ແຕ່ລະ Iteration ຈະຜ່ານຂະບວນການຄົບວົງຈອນ: Planning -> Requirements -> Design -> Code -> Test -> **Evaluation** (ການປະເມີນຜົນ).

- ຈຸດສໍາຄັນຄືຂັ້ນຕອນ "Evaluation" ທີ່ທຶມງານ ແລະ ລູກຄ້າຈະມາປະເມີນຜົນງານໃນຮອບນັ້ນ ແລະ ວາງແຜນວ່າຈະປັບປຸງຫຍັງແດ່ໃນຮອບຕໍ່ໄປ.
- ມັນແກ້ໄຂບັນຫາຂອງ Waterfall ແນວໃດ?
 - ຮັບມືກັບຄວາມຕ້ອງການທີ່ບໍ່ຊັດເຈນໄດ້ດີທີ່ສຸດ: ໂມເດລນີ້ຍອມຮັບວ່າ "ລູກຄ້າອາດຈະຍັງບໍ່ຮູ້ວ່າ ຕ້ອງການຫຍັງແທ້ໆ". ການສ້າງໂຄງຮ່າງງ່າຍໆ ໃຫ້ເບິ່ງກ່ອນ ເປັນວິທີທີ່ດີທີ່ສຸດໃນການຫາຄໍາຕອບ ແລະ ປັບປຸງໄປເລື້ອຍໆ.
 - ຄວາມສ່ຽງຖືກກວດພົບໄດ້ໄວ: ບັນຫາໃຫຍ່ໆ ທາງດ້ານເທັກນິກ ຫຼື ການອອກແບບ ມັກຈະຖືກຄົ້ນພົບ ຕັ້ງແຕ່ Iteration ທໍາອິດໆ.
- ເໝາະສົມກັບໂຄງການແບບໃດ?
 - ໂຄງການຂະໜາດໃຫຍ່ ແລະ ຊັບຊ້ອນ.
 - ໂຄງການທີ່ມີຄວາມຕ້ອງການບໍ່ຊັດເຈນ ຫຼື ມີແນວໂນ້ມຈະປ່ຽນແປງສູງ.
 - ໂຄງການທີ່ມີຄວາມສ່ຽງທາງເທັກນິກໂລຢືຊູງ.
- ຕົວຢ່າງ: ໂຄງການສ້າງ "ລະບົບ Social Media"
 - Iteration 1: ສ້າງລະບົບທີ່ຜູ້ໃຊ້ສາມາດສະໝັກສະມາຊິກ ແລະ ໂພສຂໍ້ຄວາມແບບງ່າຍໆ ໄດ້ເທົ່ານັ້ນ (ບໍ່ມີຮູບ, ບໍ່ມີ Like, ບໍ່ມີ Comment).
 - Iteration 2 (Evaluation & Refinement): ພົບວ່າການໂພສແຕ່ຂໍ້ຄວາມມັນໜ້າເບື້ອ. ຈຶ່ງປັບປຸງໃຫ້ສາມາດໂພສຮູບໄດ້ ແລະ ເພີ່ມປຸ່ມ Like.
 - Iteration 3 (Evaluation & Refinement): ພົບວ່າຜູ້ໃຊ້ຢາກໂຕ້ຕອບ. ຈຶ່ງປັບປຸງໃຫ້ສາມາດ Comment ໃຕ້ໂພສໄດ້.

ຄ. ໂມເດລຕົ້ນແບບ (Prototype Model)

- ແນວຄິດຫຼັກແມ່ນຫຍັງ?
 - "ຖ້າລູກຄ້າບໍ່ຮູ້ວ່າຕ້ອງການຫຍັງ, ກໍສ້າງ 'ຕົວຢ່າງ' ໃຫ້ເຂົາເບິ່ງ!". ໂມເດລນີ້ເນັ້ນໃສ່ການສ້າງ "ໂປຣໂຕໄທ (Prototype)" ຫຼື ແບບຈໍາລອງທີ່ສາມາດໂຕ້ຕອບໄດ້ (Mock-up) ຢ່າງໄວວາ.
 - ຈຸດປະສົງຫຼັກຂອງໂປຣໂຕໄທ: ບໍ່ແມ່ນເພື່ອໃຊ້ງານຈິງ, ແຕ່ເພື່ອ "ເກັບກໍາຄວາມຕ້ອງການ" ຈາກຜູ້ໃຊ້. ຫຼັງຈາກທີ່ຜູ້ໃຊ້ລອງຫຼິ້ນໂປຣໂຕໄທໄປ ແລະ ຍືນຍັນຄວາມຕ້ອງການແລ້ວ, ໂປຣໂຕໄທນັ້ນມັກຈະຖືກ "ຖິ້ມ" ແລະ ທຶມງານກໍຈະເລີ່ມສ້າງລະບົບຈິງດ້ວຍໂມເດລອື່ນ.
- ມັນແກ້ໄຂບັນຫາຂອງ Waterfall ແນວໃດ?
 - ແກ້ໄຂບັນຫາ "ຄວາມເຂົ້າໃຈບໍ່ດີກັນ" ໄດ້ດີທີ່ສຸດ: ເອກະສານ 100 ໜ້າ ອາດຈະບໍ່ສື່ຄວາມໝາຍໄດ້ດີ ເທົ່າກັບໂປຣໂຕໄທທີ່ກົດໄດ້. ມັນຊ່ວຍໃຫ້ທັງທຶມງານ ແລະ ລູກຄ້າເຫັນພາບດຽວກັນ ຕັ້ງແຕ່ກ່ອນຈະເລີ່ມຂຽນໂຄດຈິງ.
- ເໝາະສົມກັບໂຄງການແບບໃດ?
 - ໂຄງການທີ່ເນັ້ນໃສ່ User Interface (UI) ແລະ User Experience (UX) ເປັນພິເສດ.
 - ເມື່ອຄວາມຕ້ອງການຂອງລະບົບບໍ່ຊັດເຈນ ແລະ ຕ້ອງການການໂຕ້ຕອບຈາກຜູ້ໃຊ້ສູງ.
- ຕົວຢ່າງ: ໂຄງການສ້າງ "ແອັບ ມືຖືສໍາລັບຜູ້ສູງອາຍຸ"
 - ຂັ້ນຕອນທີ 1: ທຶມງານໃຊ້ເຄື່ອງມື ເຊັ່ນ Figma ເພື່ອສ້າງໜ້າຈໍແອັບ ທີ່ກົດໄດ້.
 - ຂັ້ນຕອນທີ 2: ນໍາໂປຣໂຕໄທນີ້ໄປໃຫ້ກຸ່ມຜູ້ສູງອາຍຸລອງໃຊ້.

- **ຂັ້ນຕອນທີ 3:** ໄດ້ຮັບ Feedback ວ່າ: "ໂຕໜັງສືນ້ອຍເກີນໄປ", "ປຸ່ມກົດຍາກ", "ສີສັນເບິ່ງບໍ່ຊັດເຈນ".
- **ຂັ້ນຕອນທີ 4:** ທີມງານກັບໄປແກ້ໄຂໂປຣໂຕໂທ ແລ້ວນຳໄປທົດສອບໃໝ່. ເຮັດຊຳຈົນກວ່າຜູ້ສູງອາຍຸ ຈະບອກວ່າ "ໃຊ້ງ່າຍແລ້ວ".
- **ຂັ້ນຕອນທີ 5:** ເມື່ອໄດ້ການອອກແບບທີ່ເປັນທີ່ພໍໃຈແລ້ວ, ໂປຣໂຕໂທກໍຈົບໜ້າທີ່. ທີມງານຈຶ່ງເລີ່ມຂຽນໂຄດສ້າງແອັບ ຈຶ່ງຕາມແບບທີ່ໄດ້ຮັບການຍືນຍັນ.

ບົດສະຫຼຸບ:

Incremental, Iterative, ແລະ Prototype Model ຄືຄຳຕອບທຳອິດຂອງວົງການຊອບແວ ຕໍ່ກັບຂໍ້ຈຳກັດຂອງ Waterfall Model. ພວກມັນໄດ້ນຳສະເໜີແນວຄິດທີ່ສຳຄັນ ຄື **ການແບ່ງວຽກເປັນສ່ວນນ້ອຍໆ ແລະ ການຮັບຟັງຄຳຄິດເຫັນຢ່າງຕໍ່ເນື່ອງ**. ແນວຄິດເຫຼົ່ານີ້ໄດ້ກາຍເປັນຮາກຖານທີ່ສຳຄັນທີ່ສຸດ ທີ່ໄດ້ຖືກນຳໄປຕໍ່ຍອດ ແລະ ຈັດລະບຽບໃຫ້ເປັນລະບົບຫຼາຍຂຶ້ນ ຈົນກາຍເປັນ "Agile" ທີ່ເຮົາຈະໄດ້ຮຽນໃນບົດຕໍ່ໄປ.

2.4 ສະຫຼຸບພາບລວມ: ເປັນຫຍັງໂລກຈຶ່ງຕ້ອງມຸ່ງໜ້າໄປສູ່ Agile?

ຫຼັງຈາກທີ່ເຮົາໄດ້ເດີນທາງຜ່ານປະຫວັດສາດຂອງໂມເດລການພັດທະນາຊອບແວ, ຕັ້ງແຕ່ Waterfall Model ທີ່ເຄັ່ງຄັດ, ໄປຈົນເຖິງ Incremental ແລະ Iterative Models ທີ່ເລີ່ມມີຄວາມຍືດຍຸ່ນຫຼາຍຂຶ້ນ. ຕອນນີ້, ເຮົາມາເຖິງຈຸດທີ່ຈະຕອບຄຳຖາມທີ່ສຳຄັນທີ່ສຸດວ່າ: **ເປັນຫຍັງໂມເດລເຫຼົ່ານັ້ນຈຶ່ງຍິ່ງບໍ່ພຽງພໍ? ແລະ ເປັນຫຍັງໂລກຂອງການພັດທະນາຊອບແວຈຶ່ງຕ້ອງມຸ່ງໜ້າໄປສູ່ສິ່ງທີ່ເອີ້ນວ່າ "Agile"?**

ກ. ບົດຮຽນຈາກອະດີດ: ຂໍ້ຈຳກັດຂອງໂມເດລແບບດັ້ງເດີມ

ເຖິງແມ່ນວ່າ Incremental ແລະ Iterative Models ຈະດີກວ່າ Waterfall, ແຕ່ພວກມັນຍັງຄົງມີລັກສະນະຂອງ "ໂຄງການໃຫຍ່" (Big Project Mindset) ຢູ່. ພວກມັນຍັງຄົງເນັ້ນໃສ່ການວາງແຜນລ່ວງໜ້າຢ່າງລະອຽດ, ມີຂັ້ນຕອນທີ່ຊັດເຈນ, ແລະ ມັກຈະຂາດຄວາມໄວໃນການຕອບສະໜອງຕໍ່ການປ່ຽນແປງທີ່ເກີດຂຶ້ນຢ່າງກະທັນຫັນ.

ໃນຊ່ວງທ້າຍຍຸກ 1990s, ໂລກທຸລະກິດໄດ້ປ່ຽນແປງໄປຢ່າງໄວວາ. ອິນເຕີເນັດໄດ້ກາຍເປັນກະແສໜຶ່ງ, ການແຂ່ງຂັນສູງຂຶ້ນ ແລະ ຄວາມຕ້ອງການຂອງຕະຫຼາດປ່ຽນແປງໄວເປັນລາຍເດືອນ, ບໍ່ແມ່ນລາຍປີ. ໂມເດລແບບດັ້ງເດີມເລີ່ມປະເຊີນກັບບັນຫາສຳຄັນ 4 ປະການຄື:

1. ບັນຫາຂອງ "ການປ່ຽນແປງ":

- **ໂລກຄວາມຈິງ:** ລູກຄ້າອາດຈະເວົ້າວ່າ "ຂ້ອຍຢາກໄດ້ລະບົບແບບນີ້", ແຕ່ຫຼັງຈາກ 3 ເດືອນ, ຄູ່ແຂ່ງອອກ Feature ໃໝ່, ລູກຄ້າກໍອາດຈະປ່ຽນໃຈ.
- **ຂໍ້ຈຳກັດຂອງໂມເດລເກົ່າ:** ການປ່ຽນແປງຄວາມຕ້ອງການໃນລະຫວ່າງທາງ ຖືກເບິ່ງວ່າເປັນ "ສິ່ງລົບກວນ" ທີ່ທຳລາຍແຜນການ.

2. ບັນຫາຂອງ "ການສົ່ງມອບ":

- **ໂລກຄວາມຈິງ:** ທຸລະກິດຕ້ອງການປ່ອຍຜະລິດຕະພັນອອກສູ່ຕະຫຼາດໃຫ້ໄວທີ່ສຸດ ເພື່ອທົດລອງ ແລະ ຮຽນຮູ້.

- **ຂໍ້ຈຳກັດຂອງໂມເດລເກົ່າ:** ເຖິງແມ່ນວ່າ Incremental Model ຈະສົ່ງມອບເປັນສ່ວນໆ, ແຕ່ຮອບວຽນ (Cycle) ຍັງຄົງຍາວ (ອາດຈະ 6 ເດືອນຕໍ່ Increment).

3. ບັນຫາຂອງ "ການສື່ສານ":

- **ໂລກຄວາມຈິງ:** ຄວາມສຳເລັດຂອງໂຄງການຂຶ້ນຢູ່ກັບການຮ່ວມມືຢ່າງໃກ້ຊິດລະຫວ່າງນັກພັດທະນາ ແລະ ຝ່າຍທຸລະກິດ.
- **ຂໍ້ຈຳກັດຂອງໂມເດລເກົ່າ:** ມັກຈະມີການແບ່ງແຍກທີ່ຊັດເຈນ. ຝ່າຍທຸລະກິດສົ່ງເອກະສານ SRS ໃຫ້, ແລ້ວທີມພັດທະນາກໍ "ຫາຍ" ໄປເຮັດວຽກຫຼາຍເດືອນ.

4. ບັນຫາຂອງ "ຄືນ":

- **ໂລກຄວາມຈິງ:** ຜະລິດຕະພັນທີ່ດີທີ່ສຸດເກີດຈາກທີມງານທີ່ມີແຮງຈູງໃຈ, ສາມາດຈັດການຕົນເອງໄດ້, ແລະ ມີຄວາມໄວ້ວາງໃຈເຊິ່ງກັນແລະກັນ.
- **ຂໍ້ຈຳກັດຂອງໂມເດລເກົ່າ:** ມັກຈະເນັ້ນໃສ່ "ຂະບວນການ" ແລະ "ເອກະສານ" ຫຼາຍກວ່າ "ຄືນ" ແລະ "ການໂຕ້ຕອບ".

ຂ. Agile ຄືຄຳຕອບ

ໃນປີ 2001, ກຸ່ມວິສະວະກອນຊອບແວທີ່ມີປະສົບການ 17 ຄົນ ໄດ້ມາປະຊຸມກັນ. ພວກເຂົາເຫັນຖືກຕ້ອງກັນວ່າ "ຕ້ອງມີວິທີທີ່ດີກວ່ານີ້". ພວກເຂົາບໍ່ໄດ້ສ້າງໂມເດລໃໝ່ທີ່ຊັບຊ້ອນ, ແຕ່ພວກເຂົາໄດ້ຮ່ວມກັນຂຽນເອກະສານສັ້ນໆ ແຕ່ມີພະລັງທີ່ສຸດໃນປະຫວັດສາດຊອບແວ, ເຊິ່ງເອີ້ນວ່າ "ຖະແຫຼງການແຫ່ງອະຈາຍລ໌" (Manifesto for Agile Software Development).

Agile ບໍ່ແມ່ນ "ໂມເດລ" ຫຼື "ຂະບວນການ" ທີ່ຕາຍຕົວ. Agile ຄື "ຊຸດຂອງຄຳນິຍົມ ແລະ ຫຼັກການ" (a set of values and principles). ມັນຄື "ປັດຊະຍາ" ໃນການເຮັດວຽກ ທີ່ໃຫ້ຄວາມສຳຄັນກັບສິ່ງຕ່າງໆ ດັ່ງນີ້:

ຄຳນິຍົມ 4 ປະການຂອງ Agile:

ຄືນ ແລະ ການໂຕ້ຕອບ ສຳຄັນກວ່າ ຂະບວນການ ແລະ ເຄື່ອງມື
 ຊອບແວທີ່ໃຊ້ງານໄດ້ຈິງ ສຳຄັນກວ່າ ເອກະສານທີ່ຄົບຖ້ວນສົມບູນ
 ການຮ່ວມມືກັບລູກຄ້າ ສຳຄັນກວ່າ ການຕໍ່ລອງຕາມສັນຍາ
 ການຕອບສະໜອງຕໍ່ການປ່ຽນແປງ ສຳຄັນກວ່າ ການເຮັດຕາມແຜນທີ່ວາງໄວ້

ໝາຍເຫດ: Agile ບໍ່ໄດ້ບອກວ່າສິ່ງທີ່ຢູ່ເບື້ອງຂວາ (ຂະບວນການ, ເອກະສານ, ແຜນ) ບໍ່ສຳຄັນ. ແຕ່ Agile ບອກວ່າເມື່ອຕ້ອງຕັດສິນໃຈ, ໃຫ້ເຮົາ "ໃຫ້ຄຸນຄ່າ" ກັບສິ່ງທີ່ຢູ່ເບື້ອງຊ້າຍຫຼາຍກວ່າ.

ຕົວຢ່າງ:

- **ສະຖານະການ:** ທີມງານກຳລັງຈະເລີ່ມ Sprint ໃໝ່. ຕາມ "ແຜນ" ເດີມ, ຕ້ອງສ້າງ Feature A. ແຕ່ມີຂ່າວດ່ວນວ່າ ຄູ່ແຂ່ງກຳລັງຈະປ່ອຍ Feature B ທີ່ຄ້າຍຄືກັນອອກມາ.

- **ທີມ Waterfall:** ອາດຈະເວົ້າວ່າ "ບໍ່ໄດ້, ເຮົາຕ້ອງເຮັດຕາມແຜນ. Feature B ບໍ່ໄດ້ຢູ່ໃນເອກະສານ SRS."
- **ທີມ Agile:** ຈະເວົ້າວ່າ "ໂອເຄ, ຢຸດກ່ອນ. ເຮົາມາລົມກັບລູກຄ້າ (Collaboration with the customer) ເພື່ອຕັດສິນໃຈວ່າເຮົາຄວນຈະປ່ຽນແຜນ ແລະ ເຮັດ Feature B ກ່ອນບໍ່. ເຮົາຕ້ອງ **ຕອບສະໜອງຕໍ່ການປ່ຽນແປງ (Responding to change).**"

ຄ. ສະຫຼຸບ: Agile ຄືວິວັດທະນາການ, ບໍ່ແມ່ນການປະຕິວັດ

ໂລກມຸ່ງໜ້າໄປສູ່ Agile ບໍ່ແມ່ນຍ້ອນວ່າມັນເປັນແຟຊັນ, ແຕ່ມັນເປັນ **"ວິວັດທະນາການ"** ທີ່ເກີດຂຶ້ນຕາມທຳມະຊາດ ເພື່ອຕອບສະໜອງຕໍ່ສະພາບແວດລ້ອມທາງທຸລະກິດທີ່ປ່ຽນແປງໄປ. Agile ໄດ້ເອົາຈຸດດີຂອງໂມເດລເກົ່າໆ ມາລວມກັນ ແລະ ຈັດລະບຽບໃໝ່:

- ມັນເອົາແນວຄິດ **"ການແບ່ງເປັນສ່ວນນ້ອຍໆ"** ຈາກ Incremental Model.
- ມັນເອົາແນວຄິດ **"ການເຮັດຊ້ຳ ແລະ ປັບປຸງ"** ຈາກ Iterative Model.
- ມັນເອົາແນວຄິດ **"ການຮັບຟັງຄຳຄິດເຫັນ"** ຈາກ Prototype Model.

ແລ້ວນຳມາຫຸ້ມຫໍ່ໃໝ່ພາຍໃຕ້ປັດຊະຍາທີ່ເນັ້ນໃສ່ **ຄວາມໄວ, ຄວາມຍືດຍຸ່ນ, ການຮ່ວມມື, ແລະ ຄົນ**. ໃນບົດຕໍ່ໄປ, ເຮົາຈະໄດ້ລົງເລິກໃນ **Scrum**, ເຊິ່ງເປັນຂອບການເຮັດວຽກ (Framework) ທີ່ໄດ້ຮັບຄວາມນິຍົມທີ່ສຸດໃນການນຳເອົາປັດຊະຍາ Agile ມາປະຕິບັດໃຫ້ເກີດຜົນເປັນຮູບປະທຳ.